Software Deployment Performance Analysis

Prepared by: Kevin Nguyen Date: 05/06/2026 Tool Used: SQL (SQLLite/DBeaver)

Overview Analyzed [X] software deployments to identify efficiency patterns, platform coverage trends, and change type distribution across a software development pipeline.

Key Findings

Finding 1 — Deployment Trends *(Query 1):* Deployment volume grew steadily throughout 2020, peaking in February 2021 with 54 deployments. This upward trend suggests the organization scaled its release cadence following a period of platform expansion in 2020, reflecting increased development activity and an aggressive update strategy heading into 2021

Finding 2 — Idle Time *(Query 2)* On average, 59% of total deployment time is idle, meaning only 41% of the release cycle is spent on active work. This indicates significant bottlenecks in the deployment pipeline, which is likely caused by waiting periods between handoffs, environment setup delays, or approval queues. Reducing idle time is the highest-leverage opportunity to improve overall release efficiency.

Finding 3 — Platform Coverage *(Query 3)*: iOS dominated platform deployments at 24% followed by Android at 17%, together accounting 40% of all platform coverage. Xbox and Tizen represented the smallest shares at 0.02% and 0.01%, suggesting these platforms are in the early stages or low-priority targets for the organization's release. This distribution indicates a mobile- first deployment approach with limited investments in gaming.

Finding 4 — most complex deployments *(Query 4)* All maximum-complexity deployments (8 platforms) show significant duration variance, ranging from 74 to 321 units. The 4x difference between the fastest and slowest full-platform deployments suggests that platform count alone does not determine release time; other factors, such as change type complexity or environment issues, likely contribute. BAT09627 is a notable outlier at 321 units and warrants further investigation.

Finding 5 — **change type breakdown** *(Query 5)* Design changes were the most frequent driver of deployments at 672 occurrences, followed by store changes at 485 and configuration changes at 211. The high volume of design changes suggests the organization prioritizes frequent UI and user experience updates, while the lower configuration change count indicates backend infrastructure updates are less frequent but likely higher in complexity when they do occur.

Finding 6 — **Longest Running Deployments** *(Query 6)*The ten longest deployments ranged from 385 to 699 units in total duration. Strikingly. The longest deployment, BAT-1448, had an active duration of only 7 units out of 699 total — meaning 99% of its lifecycle was idle time. This pattern repeats across several top entries, suggesting that extended deployments are not driven by technical complexity but by prolonged waiting periods such as approval delays, environment

queues, or resource availability issues. Addressing these bottlenecks could dramatically reduce overall release cycle times.

Finding 7 — Platform Count vs Duration *(Query 7)* A clear relationship exists between the number of platforms covered and average deployment duration. Single-platform deployments averaged 92.4 units across 84 deployments, while maximum complexity deployments covering 8 platforms averaged 195.6 units — more than double the duration. The bulk of deployments fell in the 2-3 platform range with 281 and 44 occurrences respectively, suggesting most releases target a core set of platforms rather than full cross-platform rollouts. This correlation confirms that platform complexity is a key driver of deployment time.

Finding 8 — High Complexity Deployments *(Query 8)* 78 deployments required all three change types simultaneously — design, configuration, and store changes. These high-complexity deployments represent the most resource-intensive releases in the pipeline and are the most likely candidates for extended duration and elevated idle time. Establishing a dedicated review and approval process specifically for these deployments could significantly reduce bottlenecks and improve overall release predictability.

Recommendations

1. **Investigate idle time** during multi-platform deployments to identify and eliminate bottlenecks in handoff and approval processes
2. **Consider staggered release strategies** for high-platform-count deployments to distribute workload and reduce peak idle time
3. **Standardize change bundling** to reduce the frequency of triple-change deployments and lower pipeline complexity

---

Conclusion

This analysis highlights significant opportunities to streamline the deployment pipeline and reduce total release duration. With 59% average idle time and a clear correlation between platform complexity and deployment length, the greatest gains can be achieved by targeting approval bottlenecks and introducing structured release strategies for high-complexity deployments.